CS6600_18_PA3

Deepak Charan S, Agnideb Mukherjee, Sooryanarayanan G

1 Introduction

This Assignment extends the Assignment-2 processor with a TAGE_SCL branch predictor and the pipeline control required to execute along the predicted path.

Deepak had done the TAGE core, Sooryanarayanan had written the statistical corrector and Agnideb finished the Loop Predictor. All 3 of us worked together to complete the final set up

2 TAGE-SC-L Predictor Organization

We implemented the standard TAGE-SC-L predictor as described in the references. The predictor uses multiple prediction components with different history horizons rather than depending on one monolithic table.
TAGE supplies tagged long-history predictions, the statistical corrector (SC) changes the result when its history-based evidence is useful, and the loop predictor handles highly regular counted loops. A separate branch target table (BTB) identifies branch PCs and supplies their targets.

2.1 Base and Tagged TAGE Tables

- The base predictor is a 1024-entry bimodal table. Each entry is a 2-bit saturating direction counter, so the base table requires $1024 \times 2 = 2048$ bits. It provides a prediction when no tagged table matches and acts as the alternate prediction for tagged entries.
- The tagged component contains four tables, each with 256 entries. Their history lengths are 8, 16, 32, and 64 branch outcomes respectively.
- Each entry contains a valid bit, a 9-bit tag, a 3-bit signed prediction counter (whose enforced range is $[-4, 3]$), and a 2-bit useful counter(in $[0, 3]$). Hence, overall an entry requires 1+9+3+2=15 bits.
- Thus each tagged table uses $256 \times 15 = 3840$ bits and combined, they use $3840 \times 4 = 15360$ bits.
- Longer histories distinguish branches with similar short-term behavior, while shorter histories provide more robust predictions when long histories are not yet trained.
- At prediction time, the tables are indexed using a hash over the PC combined with the history. Tags use a separate hash from the index as well.
- The longest-history matching entry is the provider and the next matching entry is the alternate.

- The provider supplies the primary TAGE direction; the alternate or bimodal entry is retained as a fallback (and for update operations).
- The useful bits control replacement and allocation: an entry that is not useful can be replaced when a misprediction requires a longer-history entry. Allocation uses the saved folded histories from prediction time so that training remains consistent even after the global history changes.

2.2 Branch Target Table

- The BTB has 1024 direct-indexed entries. Each entry stores a valid bit, a 32-bit full PC, a 32-bit target, and a 2-bit branch type, for 67 bits per entry, cumulating up to a total of 68608 bits.
- The BTB serves two purposes:
  1. It identifies whether a fetched PC is currently known to be a branch.
  2. It supplies the target used for taken predictions.
- A BTB miss predicts not-taken/sequential execution and is later trained when the branch resolves.

2.3 Folded Histories and Shared Control State

- The predictor maintains 64 bits of global outcome history and 16 bits of path history.
- The global history records taken/not-taken outcomes, while the path history records recent PC-path information.
- Folded versions of the global history are stored in prediction metadata for the four tagged-table history lengths.
- The predictor also contains an 18-bit usefulness-aging counter, and one phase bit. The aging phase periodically clears alternating usefulness bits so that old entries eventually become replaceable.
- These shared structures overall require $80 + 32 + 19 = 131$ bits.

2.4 Statistical Corrector

A Statistical Corrector is integrated with the TAGE branch predictor to improve accuracy by correcting TAGE predictions. The SC in our implementation consists of six tables: two bias tables and four Geometric History Length (GeHL) tables. Each table contains 128 entries. Each entry is a signed 6-bit saturating counter, ranging from -32 to +31. The GeHL tables use history lengths of 4, 8, 16, and 32.

For each prediction, the SC obtains an index using the program counter, folded history, and the produced prediction TAGE (via **GetSCIndex()**). An SC-sum is obtained by adding all the counter values from the tables using the index (via **GetSCSum()**). An additional bias of ± 12 is added depending on whether the TAGE predicts `taken` or `not taken`. The SC prediction is `taken` when the sum is non-negative.

The SC do not always override the TAGE prediction. It is used as a correction mechanism rather than as a replacement for TAGE. A threshold is used to identify the low-margin SC predictions.

It's set to 16 in this. When the SC disagrees with a strong TAGE prediction and has a low margin, we use a three-bucket choice counter corresponding to different ranges of SC magnitude. Each choice counter determines whether SC or TAGE should be selected for the final prediction.

The training is done when SC predicts incorrectly or has a low margin. The tables for conditional branches are updated towards the actual branch outcome using the above-mentioned saturating counters. The choice counters are also updated based on SC predictions and SC-TAGE conflicts.

2.5 Loop Predictor

The loop predictor contains 64 direct-indexed tagged entries. Each entry stores a valid bit, a 10-bit tag, a 10-bit current iteration count, a 10-bit learned trip count, a 4-bit confidence counter, a 4-bit replacement-age value, and a direction bit. The total logical storage is

$$64(1 + 10 + 10 + 10 + 4 + 4 + 1) = 2560 \text{ bits.}$$

The predictor learns the continuation direction and trip count from resolved conditional branches. Continuation outcomes advance the current count. At an exit, a matching trip count increases confidence; a mismatch updates the trip count and resets confidence. The current count resets after the exit. Replacement age protects trained entries against colliding branches. Counts are bounded by 1023, while confidence and replacement age saturate at 15.

A loop prediction is selected only when the entry is valid, its tag matches, the trip count is nonzero, confidence is at least 7, and the current count does not exceed the learned trip count. It predicts continuation before the learned exit and the opposite direction at the exit. Bit 63 of the saved prediction metadata records loop selection. Separate counters track selection, correctness, and table reads and writes. Reset clears the loop entries and these counters, and logical storage accounting includes the loop table alongside TAGE and SC.

2.6 Prediction and Update Flow

The prediction procedure is as follows:

1. The predictor increments `lookups` and folds the current global history at lengths 8, 16, 32, and 64. These folded values are saved in the prediction metadata so that a later update can reconstruct the allocation candidates.

2. The 1024-entry BTB is indexed by bits of the PC. If the entry is invalid or its stored full PC does not match, the predictor increments `branches_not_found`, predicts not-taken, and returns the sequential PC. This is a prediction lookup even though no branch was found.

3. On a BTB hit, the predictor increments `branches_found` and retains the BTB target. The 1024-entry base counter is read; values 2 and 3 predict taken, while values 0 and 1 predict not-taken. The base counter is also used as the initial alternate prediction and its strong states are used as a confidence signal.

4. The four tagged tables are searched from longest history to shortest history. The first matching entry is the provider and the second is the alternate. A tagged counter in the range $[0, 3]$ predicts taken and a counter in the range $[-4, -1]$ predicts not-taken. A provider counter at -4 or 3 is treated as strong; otherwise the base/provider confidence is weaker. The provider index, tag, provider direction, alternate direction, and validity bits are packed into the metadata.

5. For a conditional branch, the six SC counters are read and summed with a bias based on the TAGE direction. The sign of the sum gives the SC direction, and its magnitude is compared with the threshold 16. SC can override TAGE only when the two directions disagree, TAGE is strong, and the SC margin is low. The low-margin case is mapped to one of three choice buckets, whose signed chooser counter decides between SC and TAGE.

6. The loop table is then read for a conditional branch. A valid matching entry with a nonzero trip count, confidence at least 7, and a current count within the learned trip count can override the direction. The final loop iteration predicts the opposite of the learned continuation direction; other iterations predict continuation.

7. The final direction is taken from the selected loop result, SC result, or TAGE result, in that order of refinement. An unconditional jump is always taken and uses its BTB target; SC and loop refinement are skipped for unconditional jumps. The predictor increments either `predicted_taken` or `predicted_not_taken`.

The update procedure uses the saved metadata and the actual resolved result:

1. `updates` is incremented once. The predictor classifies the result as correct, a direction error, or a target error. Direction is checked first; the target is checked only for a correctly directed taken prediction, so one branch cannot increment both error counters.

2. The resolved PC, target, and branch type are written into the BTB. This makes a cold-start branch recognizable on a later lookup.

3. For conditional branches, the loop entry is trained. Continuations advance the current count; a direction change learns or verifies the trip count, raises confidence for repeated agreement, and adjusts the replacement age.

4. If the SC choice bucket was used, its 5-bit signed chooser is incremented when SC was correct and decremented when SC was wrong, subject to its range $[-16, 15]$. The six SC counters are trained toward the actual outcome when SC is wrong or the SC margin is low, saturating at $[-32, 31]$.

5. On a BTB miss, the PC-indexed base counter is trained toward the actual direction. On a tagged hit, the provider counter is trained toward the actual direction and its useful counter is adjusted only when provider and alternate disagree. The provider entry is checked again using its saved tag before it is modified, preventing history changes from updating a replacement entry incorrectly.

6. If the actual outcome differs from the original TAGE direction, one candidate longer-history tagged entry may be allocated. Candidate entries are weighted by history length; invalid or not-useful entries are preferred, and at most one entry is allocated per misprediction.

7. Usefulness aging advances after every update. When its interval expires, alternating useful-bit planes are cleared. Finally, the resolved PC and actual outcome are appended to path and global history for future predictions.

3 Direction and Target Error Accounting

- All branch statistics were collected for the selected predictor and were updated at well-defined pipeline events.
- A `lookup` occurs when the fetch stage asks the branch-prediction system for a prediction at a fetch PC. Therefore, lookup counters include every fetched PC, including PCs containing arithmetic, memory, and halt instructions.
- A predictor update occurs only when a conditional or unconditional branch reaches the control execution unit and its actual outcome and target are known.
- Each resolved branch is classified exactly once. The selected predictor first checks direction. If the predicted direction is wrong, the event increments $M_{\text{direction}}$.
- Only when the branch is predicted taken, and the direction is correct, is the target compared; an incorrect target then increments M_{target}.
- False branch identifications are reported separately because they concern a non-branch instruction, not a resolved branch. They contribute to redirect accounting but are not added to the resolved-branch total.

4 Recovery-Interval Measurement

A recovery event begins when a branch resolves and the selected prediction does not produce the correct next PC. The simulator then redirects the next PC, squashes younger fetch and pipeline work, removes younger checkpoints, and records one recovery event.

For a true elapsed recovery interval, the cycle-level measurement is

$$C_{\text{recovery interval}} = C_{\text{first correct path fetch}} - C_{\text{branch resolution}}. \tag{1}$$

The current statistic named `branch.recovery_cycles` does not store this latency. It records the number of recovery events, so it should be interpreted as a recovery-event count unless additional timestamps are added. The related squash counters measure the work removed by recovery: `squashed_fetches`, `squashed_pipeline_operations`, `squashed_completions`, and `squashed_data_requests`.

5 IPC and MPKI

Let I_{retired} be the number of retired instructions and C_{cycles} the total cycle count. Instruction throughput is

$$\text{IPC} = \frac{I_{\text{retired}}}{C_{\text{cycles}}}. \tag{2}$$

Using the required direction and target error counters, branch mispredictions per thousand retired instructions are

$$\text{MPKI} = \frac{1000\,(M_{\text{direction}} + M_{\text{target}})}{I_{\text{retired}}}. \tag{3}$$

6 Speculation Control and Checkpoint Recovery

The speculation controller maintains unresolved branch checkpoints identified by dynamic instruction sequence number. Its configured checkpoint limit must be positive. Reset clears the checkpoint collection, and fetch is permitted while the collection has space. An instruction is speculative when an unresolved checkpoint has a smaller sequence number. Its speculation depth is the number of such older checkpoints.

6.1 Speculative Dispatch Checks

The dispatch check allows non-speculative instructions to proceed. For a speculative instruction, the permitted opcodes are `ADD`, `SUB`, `AND`, `OR`, `XOR`, and `ADDI`. Other opcodes wait until they are no longer speculative. This supplies the controller-side dispatch restriction used by the pipeline; execution and retirement remain part of the shared processor integration.

6.2 Checkpoint Resolution

Branch resolution locates the checkpoint using the dynamic instruction sequence number. The correct next PC is the branch target when taken and otherwise the branch PC plus four. Comparing this value with the checkpoint's predicted next PC determines whether recovery is required. The returned recovery decision records the branch sequence, the correct next PC, and whether the prediction was wrong. Resolving a sequence without a checkpoint raises an error rather than silently choosing another checkpoint.

The resolving checkpoint is removed. During recovery, `RemoveYoungerCheckpoints` removes checkpoints whose sequence numbers are greater than the resolving branch's sequence number, while preserving older unresolved branches. These helpers support the machine's redirect and squash operations described above.

7 Inherited Cache-Controller Completion

The cache support used by Assignment 3 completes the inherited controller functionality in the `submission3` source tree. The completed functions cover local address mapping, replacement selection, pipeline backpressure, arbitration, miss tracking, writeback handling, and cycle-level grant application.

7.1 Address Mapping and Replacement

Local address mapping uses the controller's own line size and set count. For address a, line size L, and set count N, the byte offset is $a \bmod L$, the block base is $a-(a \bmod L)$, the set is $\lfloor a/L \rfloor \bmod N$, and the tag is $\lfloor a/L \rfloor / N$ using integer division. Victim selection respects reserved ways and the configured replacement policy. The Tree-PLRU `DebugState` helper returns the current replacement bits for the requested set without changing them.

7.2 Three-Stage Backpressure and Arbitration

`StageIntent` plans INDEX, LOOKUP, and OUTPUT backpressure from current state and resolved grant flags without mutating the controller. A lower response is accepted only for a recognized, correctly typed, echo-valid owner that is in flight. A demand fill uses the combined LOOKUP port

for that cycle. Upper-request acceptance reflects whether INDEX is already free or can advance into LOOKUP during the cycle.

`OutputWillDrain` and `DecideLookup` determine whether existing pipeline work can progress. Response selection considers OUTPUT, ready MSHR waiters, and acknowledged writeback waiters in arrival order, with response ID as a tie-breaker. Lower-request selection considers unsent demand misses and writebacks, prevents demand requests from bypassing overlapping incomplete writebacks, and applies writeback pressure and bounded-bypass rules. `ResponseIntent` and `LowerRequestIntent` expose the selected candidates without allocating new IDs or modifying state.

7.3 Hits, Misses, and Ordered Fill Replay

Hit handling constructs the required read data or acknowledgement and applies writes to the cache line when appropriate. Waiter construction retains request identity and ordering information. A compatible miss can append a waiter to an existing MSHR; a primary miss allocates an MSHR and reserves its chosen way. Capacity checks cover MSHR entries, waiter slots, and eligible ways. A dirty victim requires writeback capacity before the miss commits.

`ApplyLowerResponse` validates the response owner and echoed request fields. A demand fill installs the complete local line and replays its waiters in arrival order, so a later load observes an earlier store within the same merged miss. The filled way remains reserved until all waiter responses have been granted. A writeback acknowledgement moves its owner to the acknowledgement-ready state.

7.4 Writebacks, Grants, and Statistics

`PlanWriteback` computes the transitive overlap closure of mergeable unsent writebacks and checks entry and waiter capacity. Overlapping in-flight or already-granted writebacks cannot be modified. `InsertWriteback` inserts or merges the planned range while retaining byte contents and waiter ordering. `ApplyWritebackForward` connects an upper-level writeback to this handling. These checks preserve ordering between outstanding writebacks and subsequent overlapping demand requests.

`ApplyResponseGrant` marks the offered response as sent and releases its owner when all required responses have completed. `ApplyLowerRequestGrant` transitions the selected lower owner from unsent to in-flight. `CountLookupStall` records the specific reason that LOOKUP could not advance. `Operate` combines the resolved grants, accepted requests, lower responses, and permitted stage movements into next state; `Commit` makes that next state current. This maintains the current/next-state convention and avoids using newly created work as a current-cycle arbitration candidate.

8 Worked Example

The following values were collected by running `tests/one_hundred.pisa` with `multicycle_config.json` and predictor slot 0:

The derived performance values are

$$\text{IPC} = \frac{100}{731} \approx 0.1368, \tag{4}$$

$$\text{MPKI} = \frac{1000(3+0)}{100} = 30.0. \tag{5}$$

The prediction accuracy over resolved branches is

$$\frac{30}{33} \times 100 \approx 90.9\%. \tag{6}$$

Table 1: Example run statistics.

Metric	Value
Cycles, C_{cycles}	731
Retired instructions, I_{retired}	100
Resolved branches	33
Correct predictions	30
Direction mispredictions, $M_{\text{direction}}$	3
Target mispredictions, M_{target}	0
False branch redirects	0
Recovery events	3
Predictor 0 lookups	104
Predictor 0 updates	33

(PS: The lookup-based ratio 30/104 should not be reported as prediction accuracy, because the 104 lookups include non-branch instruction PCs).

9 Logical Storage and Implementation Cost

The following accounting is logical predictor storage. It describes the provisioned entries and declared field widths in the implementation; it is not a claim about physical area, power, timing, or host memory consumption. In particular, the tagged prediction counter ranges from -4 to 3 (3 bits), the useful counter ranges from 0 to 3 (2 bits), and the statistical corrector counters range from -32 to 31 (6 bits).

Table 2: Provisioned logical storage for predictor 0.

Structure	Entries	Fields per entry	Total bits
Bimodal base table	1024	2-bit prediction counter	$1024 \times 2 = 2048$
Tagged table 0	256	valid 1, tag 9, prediction 3, useful 2	$256 \times 15 = 3840$
Tagged table 1	256	valid 1, tag 9, prediction 3, useful 2	$256 \times 15 = 3840$
Tagged table 2	256	valid 1, tag 9, prediction 3, useful 2	$256 \times 15 = 3840$
Tagged table 3	256	valid 1, tag 9, prediction 3, useful 2	$256 \times 15 = 3840$
Branch target table	1024	valid 1, PC 32, target 32, type 2	$1024 \times 67 = 68608$
Statistical corrector tables	6×128	signed counter 6	$6 \times 128 \times 6 = 4608$
Statistical corrector choice	3	signed choice counter 5	$3 \times 5 = 15$
Loop predictor	64	valid 1, tag 10, current 10, trip 10, confidence 4, age 4, direction 1	$64 \times 40 = 2560$
Global/path history	1 each	global 64, path 16	80
Allocation RNG	1	state 32	32
Usefulness aging state	1	interval count 18, phase 1	19
		Total logical storage	**93330**

The total is

$$\begin{aligned} S &= 2048 + 4(3840) + 68608 + 4608 + 15 + 2560 \\ &\quad + 80 + 32 + 19 \\ &= 93330 \text{ bits.} \end{aligned}$$

This agrees with the value returned by `PredictorOne::StorageBits()`.

9.1 Retained Prediction Metadata and Checkpoints

Prediction metadata is separate from provisioned table storage. One `BranchPrediction` record logically retains `branch_found` (1 bit), `taken` (1 bit), target (32 bits), confidence (32 bits), and the packed information field (64 bits), for 130 logical bits. A prediction set contains three such records, the selected predictor index (2 bits for values 0–2), and the next PC (32 bits):

$$S_{\text{prediction set}} = 3(130) + 2 + 32 = 424 \text{ bits.} \tag{7}$$

A branch checkpoint retains the sequence number (64 bits), predicted next PC (32 bits), and one prediction set (424 bits), for

$$S_{\text{checkpoint}} = 64 + 32 + 424 = 520 \text{ bits per checkpoint.} \tag{8}$$

The configured speculation controller allows at most 8 checkpoints, giving 4160 bits of checkpoint payload at maximum occupancy. These are transient pipeline/control records and are not included in the 93330-bit predictor storage total.

9.2 Table Reads and Writes

The following is the logical table activity per prediction or update. A conditional entry means that the access depends on a BTB hit, branch type, or the training condition (it is not silently counted as an unconditional access).

Table 3 reports the implementation's per-event logical work and bounds (not invented runtime totals).

Table 3: Logical predictor table activity.

Component	Prediction-time reads	Update-time reads/writes
BTB	1 entry read per lookup	1 entry write per resolved branch
Bimodal base	1 counter read after a BTB hit	On a conditional BTB miss, 1 counter read and 1 counter write
Tagged tables	Up to 4 indexed entry reads, one per tagged table	Provider entry read/write when valid; allocation reads candidate entries and writes at most 1 entry
Statistical corrector	Up to 6 counter reads for a conditional BTB hit	Up to 6 counter writes when SC training is enabled; choice counter read/write when its choice bucket is active
Loop predictor	1 loop entry read for a conditional BTB hit	1 loop entry read and normally 1 entry write per conditional update
Histories and aging	Folded-history computation reads shared history	Global/path history updated once per resolved branch; aging periodically writes useful bits across tagged tables